

MONARK UNIVERSITY

FACULTY OF COMPUTER APPLICATION

Course: Basics of Figma

Course Code: 166CAMJ3102

Bachelor of Computer Application - Semester III

Assignments 1 to 4 - Solutions

Assignment - 1

Q1. Differentiate between UI and UX with two relevant examples.

UI (User Interface) and UX (User Experience) are closely related but describe different parts of a product. UI is about how a product looks, while UX is about how a product feels and works for the person using it.

Point	UI (User Interface)	UX (User Experience)
Meaning	The visual look and layout of a product	The overall feel and ease of using a product
Focus	Colours, buttons, fonts, icons, spacing	User journey, satisfaction, usability, flow
Goal	Make the product visually appealing	Make the product easy and pleasant to use
Example 1	The colour and shape of a "Buy Now" button	How easily a user can complete a purchase in 3 steps
Example 2	The font and icons used in a food delivery app	How smoothly a user can search, order, and track food

In short, UI is the surface a user sees and touches, while UX is the entire experience behind it. A product can look beautiful (good UI) but still frustrate users if it is confusing to navigate (poor UX), which is why both must be designed together. A skilled designer always considers both aspects side by side, since focusing on only one leads to an incomplete and unsatisfying final product.

Q2. Explain the five stages of the design thinking process with brief justification.

Design Thinking is a user-centered problem-solving approach used to create products that truly meet people's real needs. It consists of five stages that are often repeated in a cycle rather than followed only once.

- **1. Empathize:** Understand the real users deeply through interviews and observation, so the design is based on genuine needs rather than assumptions.
- **2. Define:** Clearly state the actual problem to be solved, based on the insights gathered during empathizing.
- **3. Ideate:** Brainstorm as many possible solutions as possible, without judging ideas too early, to encourage creativity.
- **4. Prototype:** Build quick, low-cost sample versions of the best ideas, such as wireframes, to make them tangible for testing.
- **5. Test:** Show the prototype to real users, gather feedback, and refine the design based on what actually works for them.

Following these five stages ensures that a designer solves the correct problem, for the correct people, before investing heavy time and cost into full development. Because Design Thinking is non-linear, teams often revisit earlier stages, such as returning to Empathize after Testing reveals a misunderstood user need, making the whole process genuinely iterative rather than a single straight path. This flexibility is exactly what makes the method so effective for solving complex, real-world design problems.

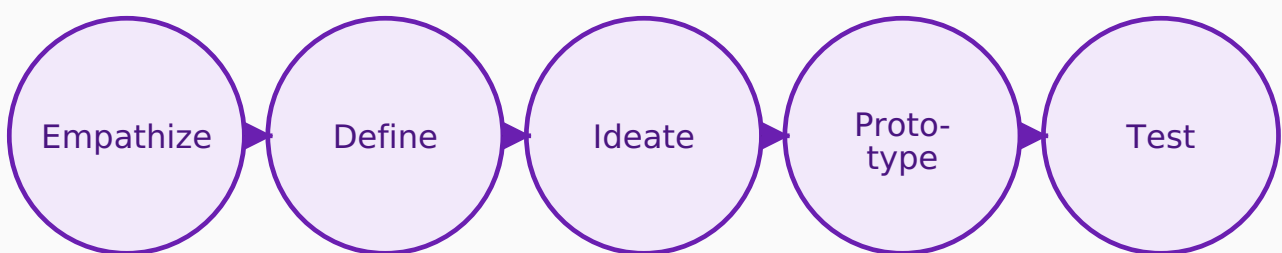


Fig: Five Stages of Design Thinking

Q3. Describe the structure and purpose of canvas, frames, and artboards in Figma.

- **Canvas:** The large, infinite workspace in Figma where all design work happens. It can be freely panned and zoomed, and it holds all the frames, shapes, and elements of a project.
- **Frames:** Rectangular containers placed on the canvas that hold and organize design elements together, similar to a page or screen boundary. Frames can be resized, nested inside each other, and are the basic building block for laying out any screen.
- **Artboards:** Figma uses the term "Frame" instead of the older term "Artboard" (used in tools like Sketch or Adobe XD), but it serves the same purpose - representing a fixed screen size, such as a mobile or desktop screen, for a specific design.

Together, the canvas acts as the open workspace, while frames (or artboards) act as defined screen boundaries within it, letting a designer work on multiple screens of an app side by side on the same canvas. This structure makes it easy to see an entire user flow at a glance, since related screens can simply be placed near each other on the same shared canvas, without needing to open separate files or pages for each individual screen.

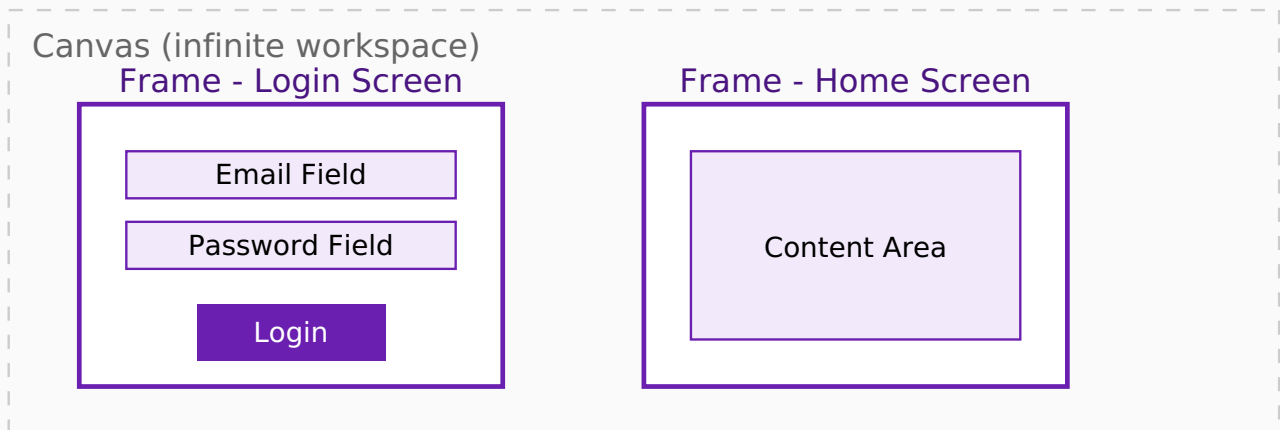


Fig: Canvas holding multiple Frames

Q4. List and explain any four basic tools available in the Figma toolbar.

- **Move Tool (V):** The default selection tool used to select, move, and resize any object or frame on the canvas.
- **Frame Tool (F):** Used to create new frames of custom or preset sizes, which act as the containers for screens and layouts.
- **Shape Tools (R, O, L):** Includes the Rectangle, Ellipse, and Line tools, used to draw basic geometric shapes that form the visual building blocks of a design.
- **Text Tool (T):** Used to add and edit text on the canvas, with full control over font, size, weight, and alignment.
- **Pen Tool (P):** Used to draw custom vector shapes and paths, useful for creating icons or unique illustrations.
- **Comment Tool (C):** Used to leave feedback notes directly on the design, helpful during team reviews.

Each of these tools can be quickly accessed using their keyboard shortcut, shown in brackets, which significantly speeds up the design workflow once a designer becomes familiar with them through regular practice. Learning these shortcuts early is one of the fastest ways for a beginner to start working efficiently inside Figma, since switching tools without reaching for the mouse saves a noticeable amount of time across a full design session.

Q5. Explain how grouping and layering improve efficiency in UI design.

Grouping

Grouping combines multiple selected elements into a single unit, so they can be moved, resized, or styled together as one object instead of adjusting each element individually every time.

Layering

Layering refers to the stacking order of elements, controlling which objects appear in front of or behind others, and organizing them in a clear, named hierarchy in the Layers panel.

Efficiency Benefits

- Reduces the time spent selecting and repositioning many separate elements one at a time.
- Makes complex designs easier to navigate and understand through a clean, organized layer structure.
- Prevents accidental changes to elements that should stay untouched while editing others nearby.
- Allows entire sections, such as a card or navigation bar, to be duplicated or moved as one clean unit.
- Makes it simple to lock specific layers in place, protecting finished work from accidental edits.

Without proper grouping and layering, even a simple design can quickly become messy and difficult to manage as more elements are added over time. A well-organized layer panel, with clearly named groups and frames, also makes it much easier for a second designer to pick up and understand a file they did not originally create, which matters greatly in team-based projects with shared files.

Q6. What is a wireframe? State its purpose and sketch a basic layout for a login screen.

A wireframe is a simple, low-detail visual blueprint of a screen, showing the basic layout and placement of elements such as buttons, text fields, and images, without focusing on colours, fonts, or final visual styling. It is often drawn in grayscale, using simple boxes and placeholder text, so that attention stays fully on structure and functionality rather than aesthetics.

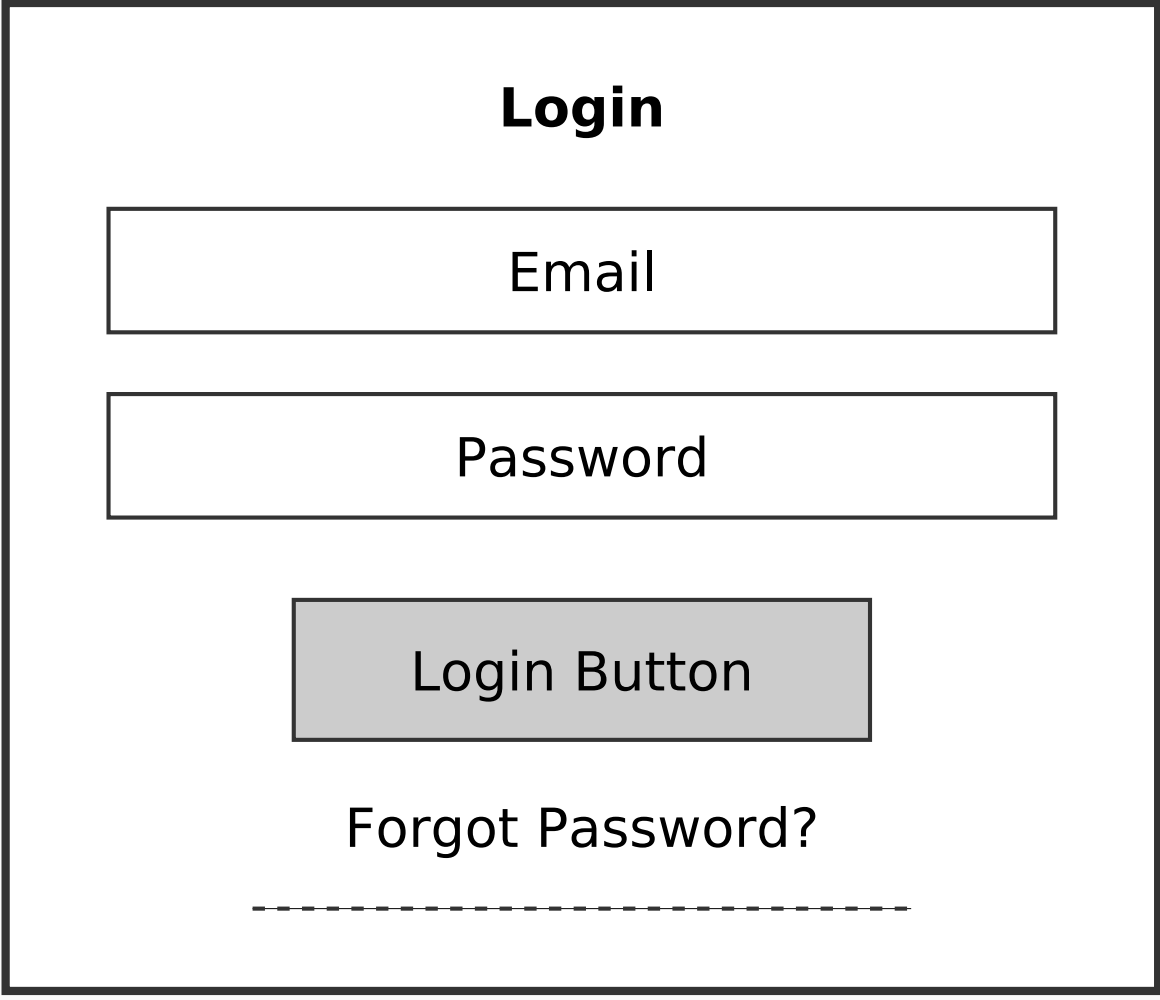
Purpose

- Helps plan the structure and layout of a screen before investing time in detailed

visual design.

- Allows quick and cheap changes to layout ideas, since no fine visual detail has been created yet.
- Helps the whole team and client agree on the basic structure early, avoiding costly redesigns later.
- Acts as a clear communication tool between designers, developers, and stakeholders during early project discussions.
- Encourages a focus on user flow and functionality before any time is spent on colours or imagery.
- Works well as a fast, low-fidelity communication tool during early brainstorming sessions with a team.

The sketch below shows a simple wireframe for a login screen, containing only the essential elements: an email field, a password field, and a login button, arranged in a clear, logical order.



The wireframe shows a login screen layout. At the top, the word "Login" is centered in a bold font. Below it, there are two input fields: "Email" and "Password", each in a rectangular box. Below the "Password" field is a "Login Button", represented by a shaded rectangular box. At the bottom, the text "Forgot Password?" is centered, followed by a dashed horizontal line.

Fig: Basic Wireframe Layout for a Login Screen

Q7. Define the role of Figma in modern UI/UX workflows.

Figma is a cloud-based design tool that has become central to modern UI/UX workflows because it combines design, prototyping, and team collaboration all within a single browser-based platform, without requiring any software installation.

- **Design Creation:** Provides all the tools needed to create wireframes, high-fidelity screens, and complete design systems.
- **Real-time Collaboration:** Multiple team members can view and edit the same file simultaneously, similar to Google Docs, seeing each other's cursors live.
- **Prototyping:** Allows designers to link screens together and simulate real app interactions before any code is written.
- **Developer Handoff:** Figma's Dev Mode lets developers directly inspect exact spacing, colours, and code snippets from the design.
- **Cross-platform Access:** Since it runs in a browser, it works seamlessly across Windows, Mac, and Linux systems.
- **Community Resources:** A large community shares free plugins, templates, and icon libraries, speeding up everyday design work.

Because of these combined strengths, Figma has largely replaced older, disconnected design and prototyping tools in most modern design teams. Its browser-based nature also means designers, developers, and clients can all view the same file simply by opening a link, without ever needing to install or purchase separate software, which lowers the barrier for teams to start collaborating quickly.

Q8. Write the steps to create a basic screen in Figma using shapes and text tools.

- **1. Create a Frame:** Press F and select a preset screen size, such as a mobile or desktop frame, from the right panel.
- **2. Add background shapes:** Use the Rectangle tool (R) to draw sections like a header bar or card background.
- **3. Add text elements:** Use the Text tool (T) to add a title, labels, and button text, adjusting font and size as needed.
- **4. Add buttons:** Draw a rectangle for the button shape, then add text on top of it, and group them together.
- **5. Arrange and align:** Use alignment options to neatly position all elements relative to the frame and each other.
- **6. Apply colours and styles:** Fill shapes with colours and adjust corner radius for a polished, finished look.
- **7. Preview the screen:** Zoom out or use the present mode to see how the finished screen actually looks.

Following these steps in order builds a screen gradually from structure to detail, which is a much more organized approach than trying to perfect every element all at once. Beginners especially benefit from this step-by-step order, since it prevents getting overwhelmed by too many decisions at the same time.

Q9. Describe the difference between a frame and a group with suitable usage examples.

Frames and groups both combine multiple elements together, but they behave quite differently and are used for different purposes in a design.

Point	Frame	Group
Meaning	A container with its own defined boundary and size	A simple collection of elements bundled together
Resizing behaviour	Can clip content that goes outside its boundary	Automatically resizes to fit its contents exactly
Auto Layout support	Supports Auto Layout for responsive design	Does not support Auto Layout
Best used for	Full screens, cards, or sections with a fixed boundary	Temporarily bundling a few related icons or shapes
Nesting	Can contain other frames, groups, and components	Can contain groups but has fewer layout controls
Example	An entire mobile screen frame	Grouping an icon and its label together

In general, frames are used for structural, screen-level containers, while groups are used for smaller, simpler bundles of elements that do not need advanced layout features. Choosing the right one matters, since overusing groups where frames are needed can make a design harder to adapt for responsive or Auto Layout-based workflows later, causing extra rework when the design needs to scale. New designers are often advised to default to frames whenever unsure, since frames offer strictly more capability.

Q10. State the advantages of using wireframes before designing a complete UI.

- **Saves time and cost:** Layout mistakes are far cheaper and faster to fix at the wireframe stage than after a fully detailed design has already been built.
- **Focuses on structure first:** Keeps the designer focused purely on layout and user flow, without getting distracted by colours or fonts too early.
- **Easier stakeholder feedback:** Clients and team members can give clear feedback on layout and functionality without being influenced by visual styling.
- **Improves usability planning:** Helps identify confusing navigation or missing elements before any real visual design work begins.
- **Encourages collaborative iteration:** Layout ideas can be quickly adjusted together with a team during early meetings.
- **Speeds up team alignment:** Developers and designers can agree on the basic screen structure early, avoiding costly rework later.

Overall, wireframing acts as a low-risk planning stage that helps catch major structural problems early, when they are still simple and inexpensive to fix. Skipping this stage often leads teams to discover fundamental layout flaws only after significant time has already been spent on detailed visuals, which is a far more costly mistake to correct, often requiring the design to be substantially reworked.

Assignment - 2 (Unit 3)

Q1. Describe the role of layout grids in maintaining alignment in a UI design.

Layout grids are sets of evenly spaced vertical and horizontal guide lines placed over a frame in Figma, helping designers position and align elements consistently across an entire screen, rather than placing them randomly by eye.

Types of Grids in Figma

- **Columns:** Vertical divisions used to align content horizontally, very common in web and desktop layouts.
- **Rows:** Horizontal divisions used to align content vertically.
- **Uniform Grid:** An evenly spaced grid of squares, often used for precise pixel-level alignment.

Role in UI Design

- Keeps spacing and alignment consistent across every screen in a project, giving

the whole product a professional, unified look.

- Makes it much faster to position new elements correctly, since they simply snap to the existing grid lines.
- Helps maintain a clear visual hierarchy, guiding the user's eye naturally through the content.
- Makes it easier to adapt a design for different screen sizes, since the grid structure defines clear, predictable zones.

Without a layout grid, designs tend to look inconsistent and unbalanced, even if each individual element looks fine on its own. A grid essentially acts as an invisible skeleton behind the entire screen, guiding every placement decision so the final result feels intentional and carefully considered rather than random.

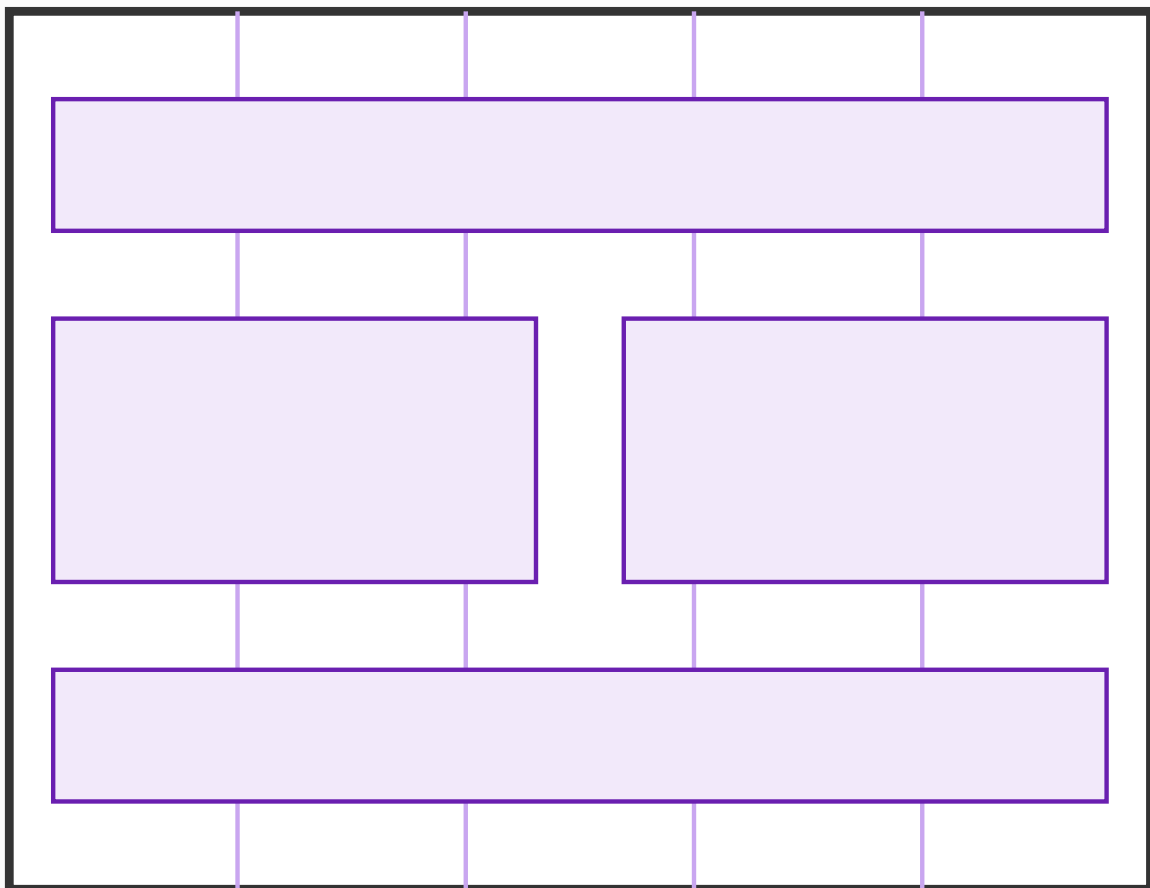


Fig: Content aligned using a column layout grid

Q2. What are constraints in Figma? Explain with suitable use cases.

Constraints define how an element inside a frame behaves and repositions itself when the parent frame is resized, ensuring elements stay correctly placed even as the screen size changes.

Types of Constraints

- **Left / Right / Top / Bottom:** Keeps the element a fixed distance from that particular edge of the frame.
- **Center:** Keeps the element centered within the frame, even as the frame is resized.
- **Scale:** Resizes the element proportionally along with the frame itself.
- **Left and Right (stretch):** Stretches the element to fill the full width between both edges.

Use Cases

- A close button set to "Top Right" stays in the corner no matter how the frame is resized.
- A background image set to "Scale" grows or shrinks proportionally with its container.
- A text input box set to "Left and Right" stretches to fill the available width on any screen size.

Constraints are essential for building UIs that adapt correctly across different device sizes without needing to manually reposition every single element by hand. Without them, resizing a frame would leave every element in its original fixed position, quickly breaking the layout on any screen size different from the original design.

Q3. Define responsive design. How is it achieved using constraints and frames in Figma?

Responsive design means designing a user interface so that it automatically adjusts and looks good across different screen sizes, from small mobile phones to large desktop monitors, without breaking the layout.

Achieving Responsive Design in Figma

- **Frames:** Different frame sizes can be created for mobile, tablet, and desktop versions of the same screen, letting a designer plan each breakpoint clearly.
- **Constraints:** Applied to elements inside a frame so they resize or reposition automatically when the frame's size changes, keeping the layout intact.
- **Auto Layout:** Combined with constraints, Auto Layout allows sections to grow, shrink, or wrap content automatically based on available space.
- **Multiple Frame Sizes:** Designing separate frames for common breakpoints, such as 375px and 1440px wide, helps verify the design holds up well at each size.

For example, a navigation bar can be set with "Left and Right" constraints so it always stretches to fill the full width of the screen, while a logo inside it stays fixed to the left using a "Left" constraint. Testing the design at multiple frame sizes early helps catch layout problems before development begins, saving significant time later in the project, and reduces the number of surprises developers encounter when actually building the responsive version.

Q4. Explain how color systems and typography enhance user readability and brand identity.

Color Systems

A well-planned color system uses a limited, consistent palette across an entire product, including primary, secondary, and accent colours, ensuring visual harmony and helping users quickly recognize interactive elements like buttons and links.

Typography

Typography refers to the careful choice of fonts, sizes, weights, and spacing used for text. Good typography establishes a clear hierarchy, guiding users naturally from headings to body text to smaller details.

Impact on Readability

- Sufficient contrast between text and background colour makes content easier to read for all users, including those with visual impairments.
- Consistent font sizes and line spacing reduce eye strain during longer reading sessions.
- Clear typographic hierarchy helps users quickly scan a page and find the information they need.

Impact on Brand Identity

- A consistent colour palette and font choice make a product instantly recognizable and memorable to users.
- Reusing the same colours and fonts across every screen builds trust and a sense of professional polish.

Together, thoughtful colour and typography choices make a product both easier to use and more memorable as a brand. Neglecting either one can make an otherwise well-structured interface feel unpolished, unprofessional, or, worse, genuinely difficult for some users to read comfortably.

Q5. What are design styles in Figma? Explain with an example of color or text style.

Design Styles in Figma are saved, reusable sets of visual properties, such as a colour, text formatting, or effect, that can be applied consistently across multiple elements throughout a project, and updated everywhere at once from a single central place.

Types of Styles

- **Color Styles:** Saved colours, such as "Primary Blue" or "Error Red", that can be applied to any fill, stroke, or background.
- **Text Styles:** Saved combinations of font family, size, weight, and line height, such as "Heading 1" or "Body Text".
- **Effect Styles:** Saved shadow or blur effects that can be reused consistently across cards or buttons.
- **Grid Styles:** Saved layout grid configurations that can be reused across multiple frames.
- **Stroke Styles:** Saved border colours and widths, keeping outlines consistent across similar elements.

Example

A "Primary Blue" colour style could be created once (e.g. #2D6CDF), and applied to every button, link, and highlighted icon in the project. If the brand colour changes later, updating this single style automatically updates it everywhere it was used, saving huge amounts of manual rework across potentially hundreds of individual elements spread throughout a large project file.

Q6. Differentiate between a component and a variant with the help of a form element example.

Components and Variants are both powerful reuse features in Figma, but they serve different purposes when building a consistent design system.

Point	Component	Variant
Meaning	A single, reusable master element that can be repeated as instances	A group of related components combined into one switchable set
Purpose	Avoids rebuilding the same element repeatedly	Manages different states or styles of the same element together
Example	A single "Text Input" component	The same Text Input with variants: Default, Focused, Error, Disabled
Switching	Not applicable, it is just one fixed design	Can be switched easily using a dropdown property in the design panel
Maintenance	Each instance updates from a single master source	All states update together when the master set changes

For a form element like a checkbox, a single Component defines its basic look, while Variants group together its "Unchecked", "Checked", and "Disabled" states, letting a designer switch between them instantly instead of managing separate, disconnected elements. This variant grouping also keeps the Layers panel far cleaner, since related states live together as one organized set rather than scattered separately throughout the file, which becomes especially valuable as a design system grows larger.

Q7. Describe the steps to create a button with hover and active variants in Figma.

- **1. Design the base button:** Create the default button using a rectangle with rounded corners and a text label, then convert it into a Component.
- **2. Duplicate for each state:** Duplicate the component to create separate versions for Hover and Active states.
- **3. Style the Hover state:** Slightly darken or change the fill colour to visually indicate the button is being hovered over.
- **4. Style the Active state:** Apply a further colour change or slight scale-down effect to show the button is actively being pressed.
- **5. Combine into Variants:** Select all the button versions and use "Combine as Variants" to group them into a single component set.
- **6. Add interactions in prototyping:** In the Prototype tab, set the "While Hovering" and "While Pressing" triggers to switch to the corresponding variant.
- **7. Test the states:** Preview the prototype and confirm each state changes correctly as expected.

This approach keeps all button states neatly organized in one place, making it very easy to maintain and update the button's appearance consistently across an entire project. Any future redesign of the button only needs to be made once inside this variant set, and it will automatically reflect everywhere the button has been used.

Button Variant Set



Fig: Button variants for Default, Hover, and Active states

Q8. Explain the process of designing an interactive multi-screen user interface.

- **1. Plan the user flow:** Sketch out the sequence of screens a user will move through to complete a task, such as browsing, adding to cart, and checkout.
- **2. Create each screen as a frame:** Design every individual screen needed for the flow, keeping spacing, colours, and typography consistent.
- **3. Build reusable components:** Create buttons, cards, and navigation bars as components so they stay consistent across every screen.
- **4. Link screens using prototyping:** In the Prototype tab, connect interactive elements, like buttons, to the screen they should navigate to.
- **5. Add transitions:** Choose appropriate transition animations, such as slide or dissolve, to make navigation feel smooth and natural.
- **6. Test the full flow:** Use Figma's Present mode to click through the entire multi-screen experience exactly as a real user would.
- **7. Refine based on feedback:** Gather comments from teammates or test users and adjust the flow before moving to development.

Designing this way ensures the final interface is not just a set of disconnected pictures, but a coherent, testable experience that closely mimics the real, finished application. This holistic approach also helps the design team spot inconsistencies in spacing or styling across screens that might otherwise go unnoticed if each screen were designed in complete isolation.

Q9. State the benefits of real-time collaboration in Figma during UI development.

- **Simultaneous editing:** Multiple designers can work on the same file at the same time, seeing each other's cursors and changes live, without needing to merge separate files afterward.
- **Faster feedback:** Team members, clients, and developers can leave comments directly on the design, speeding up the review and approval process significantly.
- **Single source of truth:** Since everyone works from the same live file, there is no confusion about which version is the most current one.
- **Remote teamwork:** Teams spread across different locations or time zones can collaborate smoothly without needing to constantly send files back and forth.
- **Reduced miscommunication:** Designers, developers, and stakeholders can all view the exact same up-to-date design at any time, reducing costly misunderstandings.
- **Faster decision making:** Live discussion during a shared editing session often resolves design questions much faster than exchanging emails.

This real-time, collaborative approach is one of the biggest reasons Figma has become so popular, replacing older workflows where designers had to manually share and merge separate static design files. It removes an entire category of version-control headaches that used to slow down design teams working together on a shared project.

Q10. Write a short note on using auto layout in interface structuring.

Auto Layout is a powerful Figma feature that automatically arranges, spaces, and resizes elements inside a frame, behaving similarly to flexbox in web development, instead of requiring a designer to manually position every single element by hand.

- Automatically adjusts spacing between elements when items are added, removed, or resized, keeping everything neatly aligned.
- Allows a frame to grow or shrink automatically based on its content, such as a button that widens to fit longer text.
- Supports both horizontal and vertical stacking directions, as well as wrapping content onto multiple lines.
- Makes designs far more responsive and reusable, since components adapt automatically instead of breaking when content changes.
- Reduces the total number of manual edits needed whenever design content or copy is updated later.

For example, a list of notification cards with Auto Layout applied will automatically stack and space themselves correctly even as cards are added or removed, saving a huge amount of manual repositioning work and greatly speeding up the overall design process. This same principle applies to buttons, form fields, and entire page sections, making Auto Layout one of the most valuable features a Figma designer can master early on.

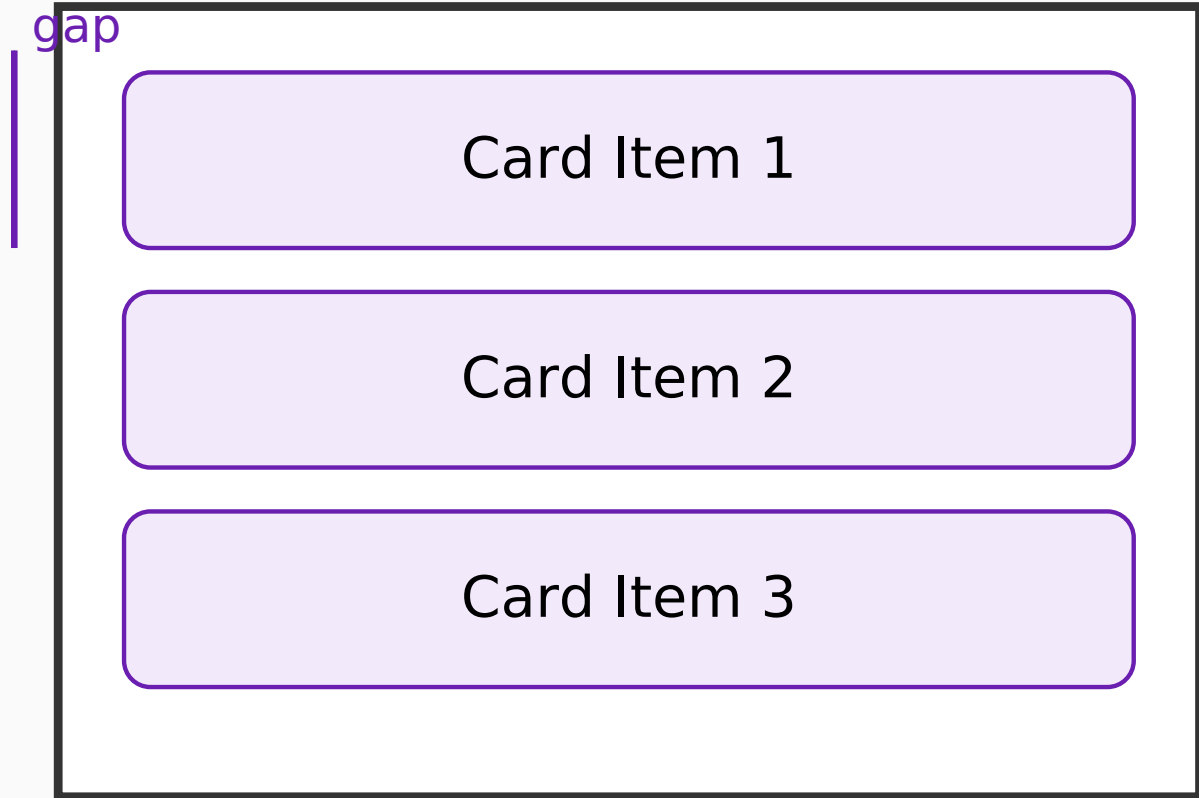


Fig: Auto Layout stacking cards with equal spacing

Assignment - 3

Q1. Define prototyping in Figma. How does it assist in evaluating design before development?

Prototyping in Figma is the process of linking static design screens together with interactive connections, called flows, so they can be clicked through and tested as if they were a real, working application, without writing any actual code.

How It Assists Before Development

- Lets designers and stakeholders experience the actual user flow, rather than just looking at isolated, disconnected screens.
- Helps catch confusing navigation or missing screens early, long before expensive development work has started.
- Allows real user testing sessions to be conducted on the prototype, gathering genuine feedback on usability.
- Gives developers a clear, interactive reference of exactly how each screen should behave and transition.
- Reduces costly late-stage changes, since major usability problems are identified and fixed while still cheap to change.
- Encourages a genuinely user-centered mindset throughout the whole design process, not only at the final testing stage.

In short, prototyping bridges the gap between a static picture and a real app, letting teams validate design decisions with genuine user interaction before a single line of code is written. This step is especially valuable for catching usability problems that are simply impossible to spot when looking at disconnected, non-clickable design images alone.

Q2. List and explain any three interaction triggers available in Figma.

- **On Click / Tap:** The most common trigger, firing the connected action when the user clicks or taps on an element, such as navigating to a new screen when a button is pressed.
- **While Hovering:** Fires only while the mouse pointer is hovering over an element, commonly used to show a button's hover state on desktop designs.
- **While Pressing:** Fires only while the mouse button or touch is actively held down, useful for showing a button's "active" or "pressed" appearance.
- **Drag:** Fires when an element is dragged, useful for simulating gestures like swiping through a carousel of images.
- **Key/Gamepad:** Fires when a specific keyboard key is pressed, useful for simulating shortcuts like pressing "Enter" to submit a form.
- **Mouse Enter/Leave:** Fires when the cursor enters or leaves an area, useful for tooltips or preview panels.

Choosing the right trigger for each interaction makes a prototype feel far closer to a real, finished application, giving much more meaningful and realistic feedback during user testing sessions. Combining several different triggers within the same prototype, such as click and hover together, can simulate genuinely complex, realistic app behaviour.

Q3. Describe the steps to link two screens using interactive transitions in a Figma prototype.

- **1. Open the Prototype tab:** Switch from the Design tab to the Prototype tab in the right-hand panel.
- **2. Select the trigger element:** Click on the button or element on the first screen that should start the navigation, such as a "Next" button.
- **3. Drag the connector:** Drag the small blue connector circle that appears from the selected element to the target second screen.
- **4. Choose the trigger type:** Select how the interaction should fire, most commonly "On Click".
- **5. Choose the transition animation:** Select an animation style, such as "Instant", "Dissolve", or "Slide In", and set its duration.
- **6. Test the link:** Use the Present button to preview the prototype and confirm clicking the element correctly navigates to the second screen.
- **7. Repeat for every connection:** Continue linking each interactive element across the whole flow until every screen is connected.

Repeating this process across every screen in a flow gradually builds a complete, clickable prototype that closely simulates how the real, finished application will actually behave once it is fully developed and deployed. Careful naming of frames also helps a lot here, since it becomes much easier to find the correct target screen while dragging connectors in a project with many screens.

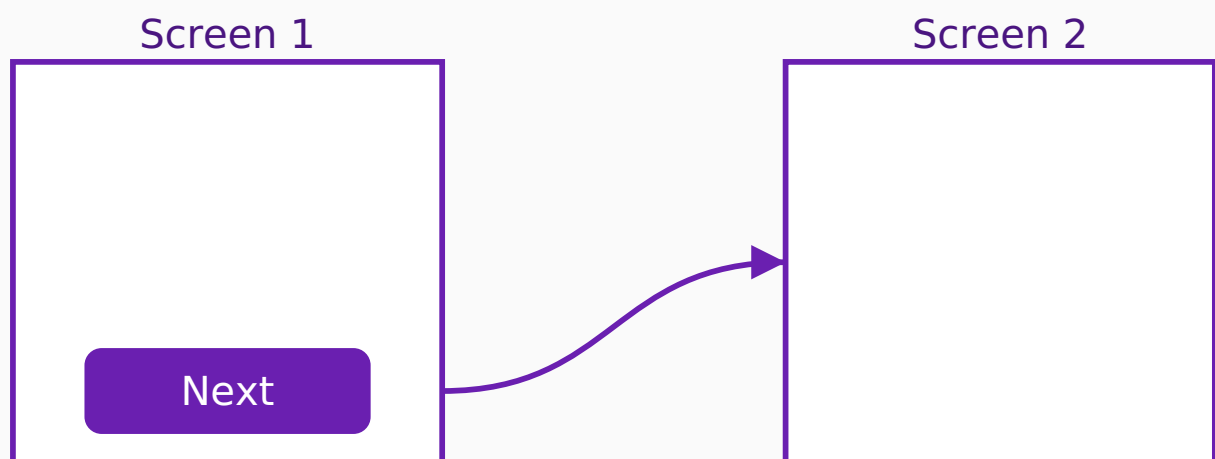


Fig: Linking Screen 1 to Screen 2 via a button interaction

Q4. What is Smart Animate? Explain with an example of a transition between cards.

Smart Animate is a special Figma transition that automatically animates the differences between two screens, such as changes in position, size, colour, or rotation, creating a smooth, natural-feeling animation instead of an abrupt jump between screens.

How It Works

- Figma compares matching layers, identified by having the same layer name, between the two connected frames.
- Any property that differs between the matching layers, like position or size, is smoothly animated from its starting value to its ending value.
- Properties such as rotation, colour, and corner radius can also be smoothly animated between matching layers.

Example: Card Transition

Imagine a small card in a grid view that, when clicked, should expand into a full-screen detail view. If both the small card and the full-screen version use a layer with the exact same name (e.g. "CardBackground"), Smart Animate will automatically animate the card smoothly growing from its small size to fill the entire screen, rather than simply switching instantly. This creates a polished, app-like feel that closely resembles animations seen in real, professionally built mobile applications, and requires no manual animation keyframing from the designer at all.

Q5. Write steps to preview and test a prototype in Figma with interaction examples.

- **1. Click the Present button:** Located in the top-right corner of the Figma editor, this opens the prototype in a full-screen preview window.
- **2. Interact naturally:** Click buttons, hover over elements, or drag components exactly as a real user would when using the finished app.
- **3. Observe transitions:** Check that screen transitions, such as sliding or dissolving, look smooth and behave as intended.
- **4. Test all flows:** Click through every possible path, such as both a "Login" and a "Sign Up" flow, to confirm every screen connects correctly.
- **5. Share for feedback:** Use the Share button to send a live link so others can test the prototype remotely on their own devices.
- **6. Note issues found:** Record any broken links, confusing flows, or missing screens discovered, and return to the Design tab to fix them.

For example, testing a shopping app prototype might involve clicking "Add to Cart", checking the cart icon updates, then clicking through to "Checkout" to confirm the entire purchase flow works smoothly from start to finish. Repeating this kind of full walk-through regularly during design helps catch broken links long before real users ever encounter them.

Q6. How are actions and transitions used to simulate real app behavior?

Actions

Actions define what should happen when an interaction trigger fires, such as navigating to another screen, opening an overlay menu, going back to the previous screen, or scrolling to a specific point.

Transitions

Transitions define how that action visually happens, controlling the animation style (such as slide, dissolve, or push) and its exact timing and easing curve.

Simulating Real Behavior

- Combining an "On Click" action with a "Slide In" transition can simulate a mobile app's screen navigation very convincingly.
- Using "Open Overlay" actions can simulate pop-up menus or modal dialogs appearing on top of the current screen.
- Combining multiple linked interactions across several screens can simulate an entire multi-step process, such as a complete checkout flow.
- Overlays combined with transitions can simulate pop-ups, toasts, or notification banners appearing briefly on screen.

By carefully combining the right actions with the right transitions, a Figma prototype can feel remarkably close to a real, finished application, even though no actual code has been written behind it yet. This realism is precisely what makes user testing on a prototype so valuable, since testers respond to it much like they would a genuine app.

Q7. State the difference between manual navigation and animated transitions in prototyping.

Both approaches connect screens together in a Figma prototype, but they differ in how the change between screens is visually presented to the user during testing, and each has a clear place depending on the stage of the project.

Point	Manual Navigation (Instant)	Animated Transitions
Meaning	Screen changes instantly with no animation	Screen change is animated smoothly over time
Visual feel	Abrupt, like a simple slideshow	Smooth and natural, closer to a real app
Best used for	Quick internal reviews, early rough prototypes	Polished prototypes intended for user testing or stakeholder demos
Performance	Faster to preview and test through many screens	Slightly slower, since each transition takes real time to play
Team use	Common in internal wireframe reviews	Common in client-facing or usability testing sessions

Early in a project, designers often use manual, instant navigation to quickly test the overall flow, then add animated transitions later once the layout and flow are finalized, to give the prototype a more polished, realistic feel. This staged approach avoids wasting time animating screens that might still change significantly during early review rounds, keeping the process efficient at every stage of the project.

Q8. How can you use hover interactions to improve button usability in Figma?

Hover interactions provide visual feedback to a user before they even click an element, confirming that the element is indeed clickable and giving a subtle preview of the resulting action, which improves overall confidence and usability.

How to Implement

- Create a "Hover" variant of the button with a slightly different fill colour, shadow, or scale compared to the default state.
- In the Prototype tab, set the button's trigger to "While Hovering" and connect it to the Hover variant.
- Optionally add a short transition duration so the colour change appears smooth rather than instant.
- Ensure the hover style change is subtle but clearly noticeable, avoiding overly dramatic colour shifts.

Usability Benefits

- Confirms to desktop users, in real time, that an element is genuinely interactive before they commit to clicking it.
- Reduces uncertainty and accidental clicks on the wrong element.
- Makes the overall interface feel more responsive, professional, and polished.

Note that hover interactions are specifically relevant for desktop and web interfaces, since mobile touchscreens do not have a true hover state available to them.

Designers building for both desktop and mobile should therefore always plan an alternative way to convey the same information to touchscreen users.

Q9. Write a short note on creating an interactive mockup for a mobile menu.

An interactive mobile menu mockup simulates how a navigation menu, often called a "hamburger menu", opens and closes within a mobile app prototype, giving a realistic preview of that core navigation experience.

Steps to Build It

- Design the main screen with a menu icon (three horizontal lines) placed in the header area.
- Design a separate overlay frame containing the full menu options, styled as a slide-in panel.
- Ensure the overlay's own layout uses Auto Layout so its menu items stay evenly spaced automatically.

- In the Prototype tab, connect the menu icon's "On Click" trigger to "Open Overlay", selecting the menu frame.
- Set the overlay's position to slide in from the left or right edge, with a smooth "Slide In" or "Move In" transition.
- Add a close button, or a tap on the background, connected back with a "Close Overlay" action to dismiss the menu.

This kind of interactive mockup lets a team experience the exact feel of the menu's animation and timing well before development begins, making it much easier to fine-tune small details like transition speed for a genuinely smooth user experience. It also gives developers a precise, testable reference for exactly how the finished menu should behave once implemented.

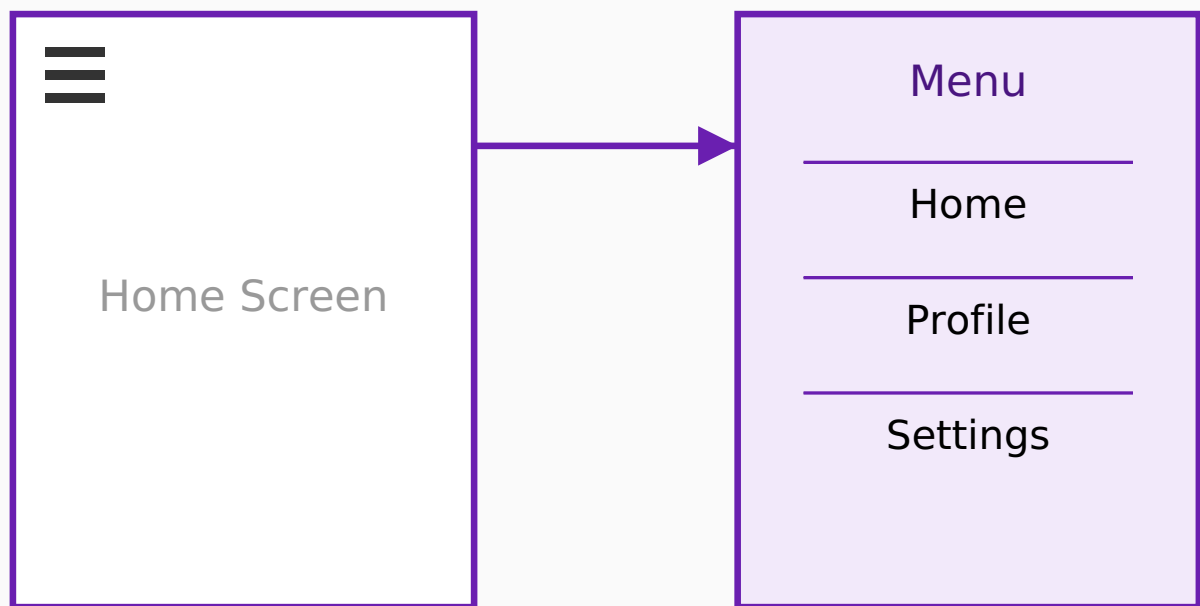


Fig: Mobile menu icon opening a slide-in overlay panel

Q10. Explain how prototyping in Figma supports feedback during the design process.

- **Realistic testing:** Users can interact with a clickable prototype almost exactly as they would with the real, finished app, producing far more accurate and useful feedback than showing static images alone.
- **Early problem detection:** Confusing navigation, missing screens, or awkward flows are discovered and fixed while changes are still quick and inexpensive to make.
- **Shareable links:** A prototype link can be shared instantly with clients, testers, or stakeholders anywhere, without requiring any special software to view it.
- **Built-in commenting:** Reviewers can leave comments pinned directly onto specific parts of the prototype, making feedback precise, clear, and easy to act on.
- **Iterative improvement:** Since Figma updates the prototype live, the design can be adjusted and retested repeatedly within the very same session.

Overall, prototyping turns design feedback from vague, subjective opinions on static pictures into specific, evidence-based insights gathered from genuine, realistic user interaction with a working simulation of the final product. This shift from guesswork to evidence is one of the most valuable contributions prototyping makes to a confident, well-informed design process, ultimately leading to a better final product for real users.

Assignment - 4

Q1. What is a design system? List its key elements and explain with an example.

A design system is a complete, organized collection of reusable components, styles, and clear guidelines that ensures visual and functional consistency across an entire product or family of products, acting as a shared source of truth for the whole team.

Key Elements

- **Design Tokens:** The smallest reusable values, such as specific colours, spacing units, and font sizes.
- **Components:** Reusable UI building blocks, such as buttons, input fields, and cards.
- **Typography and Color Styles:** A defined, consistent set of fonts and colours used throughout the product.
- **Guidelines and Documentation:** Written rules explaining how and when to correctly use each element.
- **Icon Library:** A consistent, unified set of icons used across the entire product.
- **Grid and Spacing Rules:** Standard rules defining layout structure and consistent spacing between elements.

Example

Google's Material Design is a well-known design system that defines consistent buttons, colours, typography, and spacing rules, used across many different Google products to give them all a recognizable, unified look and feel. This same idea can be applied at a smaller scale by any individual team building their own product from scratch.

Q2. Define tokens in Figma. How are they used for theming across a design?

Design tokens are named, reusable variables that store fundamental design values, such as a specific colour, spacing amount, font size, or corner radius, instead of that raw value being typed directly and repeatedly into individual elements throughout a project.

Common Types of Tokens

- **Color tokens:** e.g. "primary-500" storing a specific blue colour value.
- **Spacing tokens:** e.g. "space-md" storing a standard 16px spacing value.
- **Typography tokens:** e.g. "heading-large" storing a specific font size and weight combination.

Role in Theming

- Allows an entire product's appearance to be changed instantly by simply updating the token values, rather than manually editing every individual element.
- Makes supporting both Light Mode and Dark Mode straightforward, since each mode simply uses a different set of token values behind the same design.
- Keeps design and development perfectly in sync, since developers can use the exact same named tokens directly in their code.

In short, tokens turn a design system into something flexible and easily themeable, rather than a fixed, hard-coded set of visual choices. This flexibility is especially valuable for products that need to support multiple brands, languages, or visual themes from the exact same underlying design structure.

Q3. Explain how to create a shared component library for a team project in Figma.

- **1. Create a dedicated library file:** Set up a separate Figma file specifically to hold all shared components, kept apart from individual project design files.
- **2. Build core components:** Design essential, reusable elements such as buttons, input fields, cards, and navigation bars inside this file.
- **3. Organize with clear naming:** Use a consistent, logical naming convention, such as "Button/Primary/Large", so components are easy to find and understand.
- **4. Publish the library:** Use Figma's "Publish" feature to make these components available for use across other project files.
- **5. Enable team access:** Ensure all team members have access to the shared library through the correct team or organization settings.
- **6. Use and update centrally:** Team members insert instances of these components into their own design files; any future update made in the library file automatically updates every instance everywhere.
- **7. Maintain and version the library:** Regularly review and refine the library as the product evolves, keeping older, unused components cleaned up.

This centralized approach ensures the whole team consistently builds using the same trusted, up-to-date components, dramatically reducing design inconsistency across a larger project. It also significantly speeds up new team members getting started, since they can rely on an already-proven set of building blocks rather than creating everything from scratch themselves.

Q4. What are nested components? Explain with an example of a navigation bar.

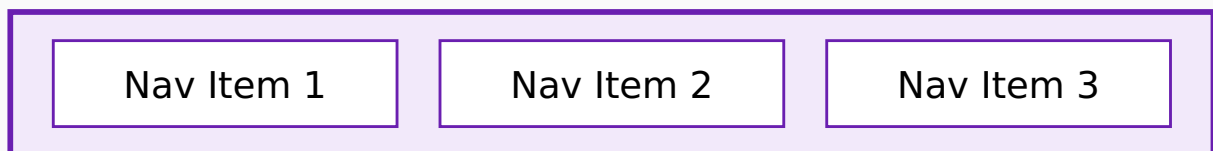
Nested components are components that are built by combining other smaller components together inside them, creating a layered structure where a change to an inner component can automatically update every larger component that contains it.

Example: Navigation Bar

- A "Nav Icon" component is created first, as a small, individual reusable element.
- A "Nav Item" component is then built by nesting the Nav Icon together with a text label.
- Finally, a full "Navigation Bar" component is built by nesting multiple Nav Item components together in a row.
- Each Nav Item can also expose its own properties, such as an "active" state, controlled from the parent Navigation Bar.

With this nested structure, updating the base "Nav Icon" component, such as changing its colour or style, automatically updates every Nav Item and every Navigation Bar built using it throughout the entire project. This layered approach keeps a design system flexible, scalable, and consistent, since large, complex elements are always built from smaller, well-tested, and independently maintained building blocks, rather than being duplicated and rebuilt separately each time.

Navigation Bar Component



+ "Home" text = Nav Item

Fig: Nested Components - Icon + Text = Nav Item, Nav Items = Navigation Bar

Q5. Describe advanced use of auto layout for designing a dynamic pricing card layout.

A pricing card must adapt cleanly to varying content, such as different plan names, feature list lengths, or price values, which makes it an excellent real-world use case for Figma's advanced Auto Layout features.

Advanced Auto Layout Techniques

- **Nested Auto Layout:** The overall card uses vertical Auto Layout, while its internal feature list uses its own separate vertical Auto Layout, allowing each section to grow independently.
- **Fill vs Hug settings:** The card's width can be set to "Fill" its parent container, while its height is set to "Hug" its contents, so it grows automatically as more features are added.
- **Padding and gap control:** Consistent internal padding and spacing between elements, like the price, features, and button, are maintained automatically as content changes.
- **Responsive button:** The "Choose Plan" button can be set to stretch and fill the card's full width, whatever that width happens to be.

With these settings combined, a single, well-built pricing card component can automatically resize correctly for a "Basic" plan with 3 features or a "Premium" plan with 8 features, without any manual adjustment ever being needed. This same principle scales naturally to an entire pricing page with several cards shown side by side, keeping them all visually consistent regardless of how much content each one contains.

Q6. State accessibility design principles to be followed while designing for web applications.

- **Sufficient color contrast:** Ensure text has enough contrast against its background so it remains readable for users with low vision, following WCAG contrast guidelines.
- **Do not rely on colour alone:** Important information, such as form errors, should also be indicated using icons or text, not colour alone, for colour-blind users.
- **Readable font sizes:** Use text sizes that remain legible without requiring users to zoom in manually.
- **Clear focus states:** Interactive elements should show a clearly visible focus outline for users who navigate using only a keyboard.
- **Adequate touch target size:** Buttons and links should be large enough to tap accurately, especially important on mobile devices.
- **Descriptive labels:** Form fields and icons should have clear, descriptive labels so screen readers can announce them properly.
- **Logical reading order:** Structure content so it flows in a sensible order for assistive technologies to read aloud.

Designing with accessibility in mind from the very start ensures the final product is genuinely usable by the widest possible range of people, including those with visual, motor, or cognitive impairments. Treating accessibility as an afterthought, rather than building it in from the beginning, usually results in a far more difficult and expensive retrofit later in the project.

Q7. How does Figma Dev Mode help developers during handoff?

Dev Mode is a specialized mode in Figma designed specifically for developers, providing a clean, focused view of a design's technical details, making the handoff from design to development far smoother and more accurate than working from a regular design view.

Key Features

- **Exact measurements:** Automatically shows precise spacing, padding, and sizing values between elements, without requiring manual measurement.
- **Code snippets:** Generates ready-to-use CSS, iOS, or Android code snippets directly from the selected design element.
- **Asset export:** Lets developers directly export images and icons in the exact required format and resolution.
- **Style inspection:** Clearly displays the exact colour codes, font details, and effect values used in the design.
- **Component documentation:** Shows any notes or usage guidelines a designer has attached to a specific component.
- **Change tracking:** Highlights what has changed in the design since a developer last checked, avoiding missed updates.

By providing this direct, accurate technical information, Dev Mode significantly reduces the back-and-forth communication and guesswork that traditionally happens between designers and developers during handoff. It allows developers to work with far more confidence, since they can trust the values shown are exact rather than approximated from a static image.

Q8. Write the steps to export assets and code snippets from a Figma design.

- **1. Select the element:** Click on the specific shape, icon, or image layer that needs to be exported.
- **2. Open the Export panel:** Scroll to the bottom of the right-hand Design panel and click the "+" next to the Export section.
- **3. Choose format and size:** Select the desired file format, such as PNG, JPG, or SVG, and the correct export scale (e.g. 1x, 2x, 3x for different screen densities).
- **4. Click Export:** Click the Export button to download the chosen asset directly to the computer.
- **5. Switch to Dev Mode for code:** Open Dev Mode and select the same element to view its properties.
- **6. Copy the code snippet:** Choose the target platform (CSS, iOS, or Android) and click "Copy" to copy the generated code snippet directly into a code editor.
- **7. Verify in the codebase:** Paste the exported code into the project and confirm it renders correctly, matching the original design.

This straightforward export process ensures developers receive pixel-accurate assets and ready-to-use code, closely matching exactly what was designed, with minimal manual guesswork required. It also saves considerable development time, since assets no longer need to be manually recreated or measured by hand from a static reference image.

Q9. Explain the importance of version control and file organization in collaborative projects.

Version Control

Figma automatically saves a complete version history of every file, and designers can also manually save named checkpoints, allowing the team to review past changes or safely restore an earlier version if something goes wrong.

- Prevents permanent loss of work if a mistake is made or unwanted changes are accidentally saved.
- Allows the team to clearly see who made a particular change, and exactly when it was made.

File Organization

Good file organization means using clear project folders, consistent page names, and a logical, well-structured layer hierarchy within every file.

- Makes it much faster for any team member to find the specific screen or component they need to work on.
- Reduces confusion and accidental duplicate work in larger, more complex team projects.
- Helps new team members onboard quickly, since they can navigate an unfamiliar file with far less guidance needed.

Together, strong version control and clean file organization keep a collaborative project running smoothly, even as the number of team members, screens, and components grows significantly over time. Neglecting either one tends to cause growing confusion and wasted effort as a project scales beyond just a handful of contributors.

Q10. Briefly describe the workflow of implementing a mini-project using design system principles.

- **1. Define the design tokens:** Set up core colours, spacing, and typography values as reusable tokens or styles first.
- **2. Build base components:** Create essential reusable components, such as buttons, input fields, and cards, following the defined tokens.
- **3. Organize a component library:** Group and publish these components into a shared library file for consistent, repeated use.
- **4. Design individual screens:** Assemble full screens using instances of the library components, arranged with grids and Auto Layout.
- **5. Prototype the user flow:** Link the screens together with interactions and transitions to simulate the real, working app experience.
- **6. Test and gather feedback:** Share the prototype for testing, then refine both the components and the overall flow based on the feedback received.
- **7. Handoff using Dev Mode:** Prepare the final screens for development, exporting assets and code snippets for the development team.

Following this structured, step-by-step workflow ensures that even a small mini-project is built on a strong, consistent, and scalable design foundation, exactly the same solid approach used in much larger, real-world professional design teams. Practicing this full workflow on a small project is an excellent way for a student to genuinely understand how professional design systems work in practice.